

基于统计行为特征的加密网络流量特征刻画与分类识别

摘要

针对题目给出的 12 类已标注加密网络流量与 490 个类别未知测试数据包，本文围绕前两问建立“统计行为特征刻画—监督分类识别”的统一建模框架。针对问题一，考虑加密条件下难以直接利用明文负载信息，本文从包长、时序、方向交互与协议行为四个维度提取 33 个统计特征，并采用长度为 50、步长为 25 的滑动窗口构造局部样本；在此基础上，通过标准化类中心、欧氏距离矩阵和主成分分析（PCA）对 12 类流量的本质特征与类别差异进行系统刻画。针对问题二，在问题一特征体系保持不变的前提下，建立基于随机森林的 12 分类识别模型，并通过“窗口后验概率—包级聚合得分—Hex 唯一签名校正”的机制，将窗口级识别结果映射为数据包级标签结果。

结果表明：问题一构造的统计特征能够较好地反映加密流量的类别结构，PCA 前两主成分累计解释率达到 52.43%，且 `facebook_chat` 与 `hangouts_chat`、`bittorrent` 与 `ftps_file_transfer`、`hangouts_voip` 与 `skype_voip` 分别形成最接近的类别对，而 `aim_chat` 与 `icq_chat` 则表现出显著的稀疏时序特征。问题二的比较实验表明，随机森林的准确率达到 0.9107，Macro-F1 达到 0.8895，优于类中心最近邻与加权 KNN，并可完成测试集 490 个未知数据包的 12 类标签输出。进一步地，窗口参数灵敏度分析显示主要类别关系在参数扰动下保持稳定，说明所建模型具有较好的鲁棒性。

本文完成了问题一和问题二的建模、求解、可视化与结果分析，为问题三中面向稀疏流量的识别改进提供了可直接复用的特征基础与分类框架。

关键词：加密网络流量分类；统计行为特征；主成分分析；随机森林；灵敏度分析

目录

1 问题提出

1.1 研究背景

随着 HTTPS、TLS/SSL 等加密协议的大规模应用，传统基于明文负载的深度包检测方法在加密流量场景下受到明显限制。题目给出了 12 类加密网络流量样本，要求首先刻画其本质特征，再进一步建立识别模型完成测试集分类。因此，问题一的核心并非“直接预测”，而是回答如下三个更基础的问题：

- (1) 12 类加密流量分别具有怎样的统计行为特征；
- (2) 哪些特征能够稳定地区分不同类别；
- (3) 类别之间的相似性、差异性和潜在混淆关系是什么。

1.2 目标与思路

由于加密流量无法直接读取明文内容，本文采用“行为表征”思路，从流量外在统计规律入手，围绕以下四个方面提取特征：

- (1) **包长特征**：反映单包数据规模及其分布形态；
- (2) **时序特征**：反映包间间隔、停顿与突发性；
- (3) **方向特征**：反映上下行交互结构与方向切换；
- (4) **协议特征**：反映 TCP/UDP/TLS/SSH/STUN 及控制报文行为。

在模型方法上，本文采用滑动窗口提取局部样本，以避免仅对整份文件求平均而丢失局部结构；随后构建类中心模型、PCA 可视化模型和类别距离分析模型，最终给出问题一的系统性结论。

2 问题分析

题目所给训练数据为 12 个已知类别的加密流量文件。若直接把每个文件看作一个样本，则样本数过少，不利于分析类别内部波动与局部结构；若把单个数据包看作独立样本，则又会丢失连续通信行为。基于此，本文采用滑动窗口思想：将每个流量文件按时间顺序划分为若干局部窗口，每个窗口包含固定数量的数据包，并提取一组统计特征。

这样处理有三方面优势：

- (1) 能够从有限训练文件中生成足够数量的窗口样本，增强统计稳定性；
- (2) 保留局部时序与交互规律，避免“整文件单均值”过度平滑；
- (3) 为后续问题二、问题三中基于窗口的分类模型提供直接输入。

从业务语义角度看，这 12 类流量包含聊天、VoIP、文件传输、邮件与 P2P 等不同场景。尽管在低维空间中可能存在重叠，但它们在统计行为上理应呈现以下规律：聊天类往往具有较高方向切换率和更明显的停顿结构，VoIP 类流量更容易体现持续性与 UDP 承载特征，文件传输类则通常表现为更大的平均包长和更高的大包比例。这些判断构成本文特征构造与结果解释的出发点。

3 模型假设与符号说明

3.1 模型假设

为便于问题一建模，提出如下假设：

- H1** 同一类加密流量虽然存在局部波动，但在统计意义上具有可重复的行为模式；
- H2** 训练文件中数据包按时间顺序排列，相邻数据包能够共同表征局部通信行为；
- H3** 源地址与目的地址中出现频率更高的私网地址可近似视为本地主机，从而区分上下行方向；
- H4** 不同类别之间的主要差异可由包长、时序、方向与协议行为等外在统计量刻画；
- H5** 问题一仅使用题目给定训练样本，不引入外部数据集。

假设 H1–H4 使得在不访问明文负载的前提下，仍可通过统计规律完成类别刻画；假设 H5 则保证问题一的分析完全基于题目数据完成。

3.2 符号说明

表 1: 主要符号说明

符号	含义
W	一个滑动窗口，包含若干连续数据包
m	窗口长度，即窗口内数据包个数，本文取 $m = 50$
s	滑动步长，本文取 $s = 25$
t_i	窗口内第 i 个数据包的到达时刻
L_i	窗口内第 i 个数据包长度
Δt_i	相邻数据包到达间隔， $\Delta t_i = t_{i+1} - t_i$
d_i	第 i 个数据包方向，取值为 +1（上行）或 -1（下行）
$\mathbf{x}_{k,j}$	第 k 类第 j 个窗口的特征向量
$\boldsymbol{\mu}_k$	第 k 类流量的标准化特征中心
D_{ab}	第 a 类与第 b 类流量中心间的欧氏距离

4 模型建立与计算

4.1 数据预处理与滑动窗口构造

本文优先使用题目配套的 `xlsx` 文件进行建模，因为其与 `pcap` 文件信息一致且更便于程序读取。读取时保留 `Time`、`Source`、`Destination`、`Protocol`、`Length`、`Info`、`app_aux` 等字段，删除系统临时文件与测试文件。

设某类流量文件共含 n 个数据包，窗口长度为 m ，步长为 s ，则第 r 个窗口的索引区间可表示为

$$W_r = \{(r-1)s + 1, (r-1)s + 2, \dots, \min((r-1)s + m, n)\}.$$

当剩余数据不足一个完整窗口时，保留末尾窗口，以尽量利用全部训练信息。

4.2 统计行为特征构造

结合加密流量分析的一般做法，本文构造 33 维统计特征，分为四类：

4.2.1 包长特征

包长特征反映窗口内流量规模及分布。典型指标包括：

$$\text{len_mean} = \frac{1}{m} \sum_{i=1}^m L_i, \quad \text{large_ratio} = \frac{1}{m} \sum_{i=1}^m \mathbf{1}(L_i > 1000),$$

并进一步计算 `len_std`、`len_min`、`len_max`、`len_p25`、`len_p50`、`len_p75` 以及小包占比 `small_ratio` 等统计量。

4.2.2 时序特征

设相邻包间间隔为 $\Delta t_i = t_{i+1} - t_i$ ，则窗口持续时长、平均间隔与长停顿比例分别定义为

$$\begin{aligned} \text{duration} &= t_m - t_1, & \text{dt_mean} &= \frac{1}{m-1} \sum_{i=1}^{m-1} \Delta t_i, \\ \text{silence_ratio} &= \frac{1}{m-1} \sum_{i=1}^{m-1} \mathbf{1}(\Delta t_i > \tau), \end{aligned}$$

其中长停顿阈值取 $\tau = 0.5$ s。此外，本文采用

$$\text{burstiness} = \frac{\sigma_{\Delta t} - \mu_{\Delta t}}{\sigma_{\Delta t} + \mu_{\Delta t} + \varepsilon}$$

刻画窗口的突发性，并统计 `dt_std`、`dt_p50`、`dt_p90` 等量。

4.2.3 方向交互特征

为刻画上下行关系，设本地主机方向记为 $d_i \in \{+1, -1\}$ ，则方向切换率定义为

$$\text{switch_rate} = \frac{1}{m-1} \sum_{i=1}^{m-1} \mathbf{1}(d_{i+1} \neq d_i).$$

进一步构造上行包比例、下行包比例、上行字节占比、下行字节占比，以及有符号包长均值与标准差等特征，用以描述会话双向交互程度。

4.2.4 协议行为特征

考虑到加密流量中明文不可见，但协议头部和控制行为仍可观测，本文统计

`tcp_ratio`, `udp_ratio`, `tls_ratio`, `ssh_ratio`, `stun_ratio`,

以及 `ACK`、`SYN`、`FIN`、`Application Data` 等字段的出现比例，从而构造协议行为指纹。

4.3 标准化与类别中心模型

由于各特征量纲差异较大，需先进行标准化处理。对第 p 个特征有

$$z_{k,j,p} = \frac{x_{k,j,p} - \bar{x}_p}{s_p},$$

其中 $\bar{x}_p$ 和 s_p 分别为该特征在全部窗口样本上的均值与标准差。

对第 k 类流量的全部窗口样本，定义其标准化类中心为

$$\boldsymbol{\mu}_k = \frac{1}{n_k} \sum_{j=1}^{n_k} \mathbf{z}_{k,j},$$

其中 n_k 为第 k 类对应的窗口数量。类中心能够表征该类流量的“平均行为画像”。

4.4 PCA 可视化模型

设标准化后的样本矩阵为 $\mathbf{Z}$ ，对中心化矩阵 $\mathbf{Z}_0$ 进行奇异值分解

$$\mathbf{Z}_0 = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top.$$

则主成分得分矩阵为

$$\mathbf{Y} = \mathbf{Z}_0\mathbf{V}.$$

本文使用前两主成分对窗口样本进行二维可视化，用于考察类别分布形态与可分性。

4.5 类别距离分析模型

为了定量刻画类别之间的差异，定义任意两类流量中心间的欧氏距离为

$$D_{ab} = \|\mu_a - \mu_b\|_2 = \sqrt{\sum_{p=1}^q (\mu_{a,p} - \mu_{b,p})^2},$$

其中 $q = 33$ 为特征维数。距离越大，表明两类流量的平均行为差异越明显；距离越小，则说明这两类在统计行为上更接近，后续分类任务中更可能混淆。

5 模型结果与分析

5.1 窗口样本与核心统计特征概览

在基准参数 $m = 50, s = 25$ 下，12 类流量共提取窗口样本 2224 个。各类别窗口数量及部分代表性特征如表 ?? 所示。

表 2: 12 类流量的部分核心特征统计

标签	类别	窗口数	duration	len_mean	large_ratio	dt_p90	silence_ratio	switch_rate	tcp_ratio	udp_ratio	tls_ratio	ssh_ratio
1	aim_chat	20	49.8952	138.74	0.0040	3.1626	0.2061	0.8510	0.4780	0.0240	0.4860	0.0000
2	bittorrent	359	0.0881	750.93	0.5289	0.0060	0.0005	0.8816	0.9718	0.0000	0.0000	0.0000
3	email	79	23.8845	185.54	0.0481	0.3407	0.0517	0.5939	0.4559	0.0000	0.4711	0.0000
4	facebook_voip	319	15.1150	384.89	0.2119	0.6403	0.0315	0.6074	0.4281	0.0000	0.3251	0.0000
5	facebook_chat	240	5.7706	386.96	0.2114	0.1619	0.0311	0.5813	0.4978	0.0020	0.3860	0.0000
6	ftp_file_transfer	319	0.5535	835.04	0.5927	0.0317	0.0023	0.6977	0.8112	0.0000	0.1839	0.0000
7	hangouts_voip	199	0.2129	256.31	0.0365	0.0103	0.0001	0.4170	0.0770	0.7726	0.1155	0.0000
8	hangouts_chat	239	2.6086	358.05	0.1751	0.0843	0.0167	0.4623	0.4112	0.0000	0.4343	0.0000
9	icq_chat	31	23.3094	102.05	0.0000	1.5491	0.1587	0.8361	0.4587	0.0155	0.5258	0.0000
10	sftp_file_transfer	159	0.8878	831.85	0.5893	0.0544	0.0015	0.6812	0.3594	0.0000	0.0029	0.6350
11	skype_voip	239	0.8712	124.80	0.0008	0.0328	0.0025	0.8848	0.0428	0.9458	0.0086	0.0000
12	skype_file_transfer	23	4.6418	132.64	0.0000	0.1630	0.0222	0.4525	0.4461	0.4826	0.0139	0.0000

由表 ?? 可以直接得到若干具有解释力的现象：

- (1) **稀疏聊天类明显拉长时序尺度。** aim_chat 的平均窗口持续时长为 49.8952 s, dt_p90 = 3.1626, silence_ratio = 0.2061; icq_chat 的对应值分别为 23.3094 s、1.5491 和 0.1587。说明这两类在局部窗口中停顿更长、节奏更稀疏。
- (2) **文件传输类的大包比例与平均包长显著更高。** ftps_file_transfer、sftp_file_transfer、bittorrent 的平均包长分别达到 835.04、831.85 和 750.93 字节，大包比例分别为 0.5927、0.5893 和 0.5289，明显高于聊天与语音类。
- (3) **VoIP 类具有明显的 UDP 承载特征。** hangouts_voip 的 udp_ratio = 0.7726, skype_voip 的 udp_ratio = 0.9458，远高于多数其它类别。
- (4) **协议指纹在特定类别中很突出。** sftp_file_transfer 的 ssh_ratio = 0.6350，表现出清晰的 SSH 传输特征；facebook_voip 的 stun_ratio = 0.0548，反映出其实时通信场景中辅助协议的存在。

5.2 PCA 可视化结果

图 ?? 给出了全部窗口样本在前两主成分空间中的分布。第一主成分解释 35.16% 的总体方差，第二主成分解释 17.27%，累计解释率为 52.43%。

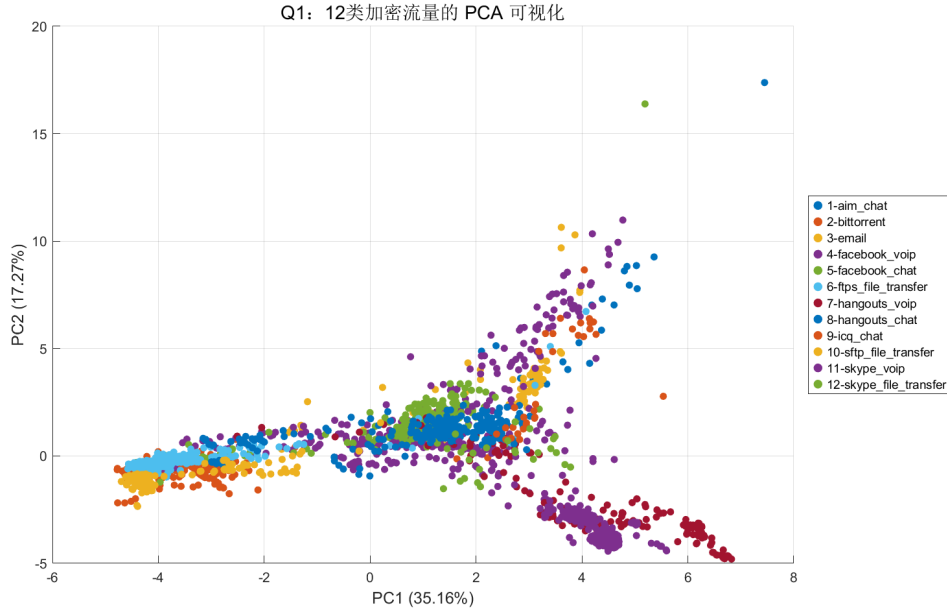


图 1: 12 类加密流量的 PCA 可视化

从图 ?? 可见，12 类流量并非完全混杂，而是呈现出“部分聚集、部分重叠、局部拉开”的结构特征，这说明本文构造的统计行为特征能够较好地保留类别差异。需要强调的是，PCA 仅为降维后的粗粒度可视化，因此部分类别在二维平面上的重叠并不意味着高维特征空间中不可分，而更应理解为：该问题存在明显的类别相近性，需要在完整特征空间中进行识别。

进一步观察可发现，聊天类与 VoIP 类在中间区域存在一定重叠，而部分文件传输类和稀疏类则沿不同方向拉伸，这与其包长分布和时序特征差异相一致。

5.3 类别间距离矩阵分析

图 ?? 为基于标准化类中心构造的类别间欧氏距离矩阵。矩阵对角线为 0，颜色越暖表示类别差异越大，颜色越冷表示类别差异越小。

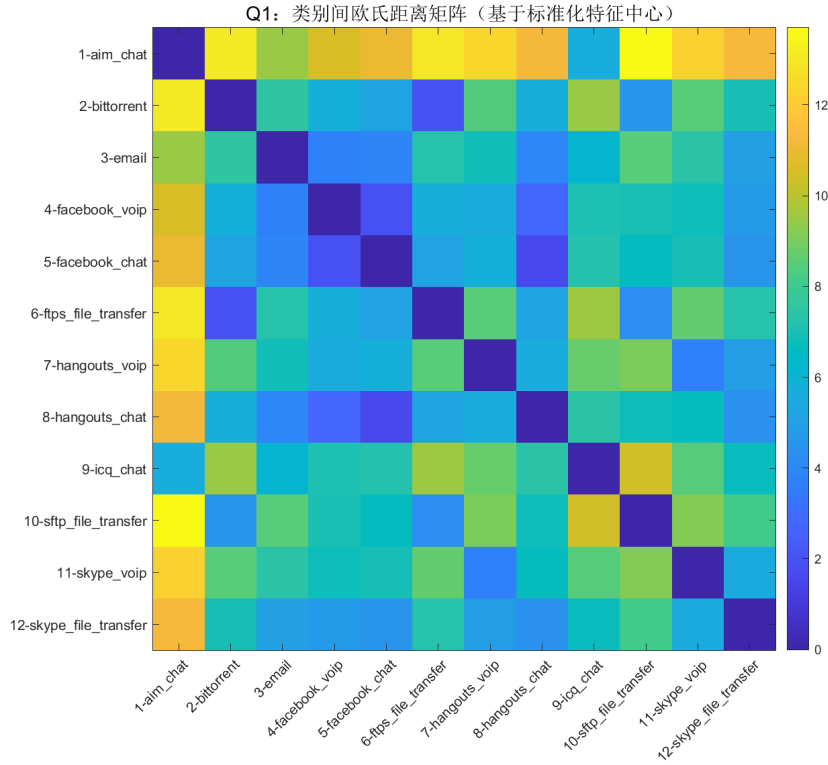


图 2: 类别间欧氏距离矩阵热力图

为便于论文分析, 表 ?? 列出了每一类流量最近的两类邻居。

表 3: 各类别最近的两类邻居

标签	类别	最近邻 1	距离	最近邻 2	距离
1	aim_chat	9-icq_chat	5.6339	3-email	9.4875
2	bittorrent	6-ftp_file_transfer	1.9539	10-sftp_file_transfer	4.5106
3	email	4-facebook_voip	3.6981	5-facebook_chat	3.9051
4	facebook_voip	5-facebook_chat	1.9605	8-hangouts_chat	2.7701
5	facebook_chat	8-hangouts_chat	1.6289	4-facebook_voip	1.9605
6	ftp_file_transfer	2-bittorrent	1.9539	10-sftp_file_transfer	4.2291
7	hangouts_voip	11-skype_voip	3.6435	12-skype_file_transfer	4.8387
8	hangouts_chat	5-facebook_chat	1.6289	4-facebook_voip	2.7701
9	icq_chat	1-aim_chat	5.6339	3-email	6.1099
10	sftp_file_transfer	6-ftp_file_transfer	4.2291	2-bittorrent	4.5106
11	skype_voip	7-hangouts_voip	3.6435	12-skype_file_transfer	5.4714
12	skype_file_transfer	8-hangouts_chat	4.3376	5-facebook_chat	4.5350

结合图 ?? 与表 ??, 可得如下结论:

- (1) 聊天类内部存在显著相近关系。facebook_chat 与 hangouts_chat 的距离仅为 1.6289, 是全体类别中最接近的一对, 表明二者在时序与协议行为上高度相似。
- (2) 文件传输类在统计层面形成一簇。bittorrent 与 ftp_file_transfer 的距离为 1.9539, sftp_file_transfer 与 ftp_file_transfer 的距离为 4.2291, 说明大包连续传输是它们共同的重要特征。

- (3) **VoIP 类之间也具有稳定相似性。**hangouts_voip 与 skype_voip 的距离为 3.6435, 说明 UDP 承载、小包占比与双向交互结构共同构成了语音类流量的行为画像。
- (4) **稀疏流量在时序维度上最为突出。**aim_chat 与 icq_chat 互为最近邻, 说明二者虽然在协议细节上不完全相同, 但都体现出明显的停顿和稀疏发送规律。

5.4 特征中心热力图分析

图 ?? 给出了各类别标准化特征中心热力图, 其中每一行对应一类流量, 每一列对应一个特征, 颜色表示该类在该特征上相对于总体均值是偏高还是偏低。

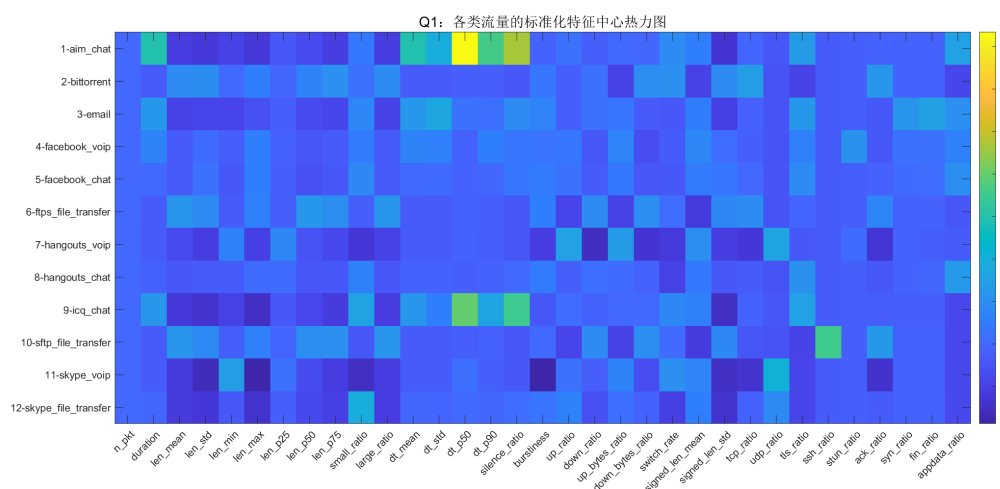


图 3: 各类流量的标准化特征中心热力图

热力图的意义在于, 它不是告诉我们“哪一类绝对最大”, 而是刻画“哪一类在什么维度上最偏离整体平均水平”。为此, 本文进一步提取了每类最显著的 5 个特征, 如表 ?? 所示。

表 4: 各类别最显著的 5 个特征

标签	类别	最显著的 5 个特征
1	aim_chat	dt_p50, silence_ratio, dt_p90, duration, dt_mean
2	bittorrent	tcp_ratio, len_p75, len_std, ack_ratio, signed_len_std
3	email	fin_ratio, dt_std, syn_ratio, tls_ratio, appdata_ratio
4	facebook_voip	stun_ratio, dt_p50, len_max, down_bytes_ratio, up_bytes_ratio
5	facebook_chat	dt_p50, len_max, appdata_ratio, signed_len_std, tls_ratio
6	ftp_file_transfer	len_mean, large_ratio, len_p50, signed_len_mean, len_p75
7	hangouts_voip	udp_ratio, small_ratio, down_ratio, up_ratio, up_bytes_ratio
8	hangouts_chat	appdata_ratio, dt_p50, switch_rate, tls_ratio, silence_ratio
9	icq_chat	dt_p50, silence_ratio, small_ratio, tls_ratio, dt_p90
10	sftp_file_transfer	ssh_ratio, len_mean, large_ratio, ack_ratio, len_p75
11	skype_voip	udp_ratio, burstiness, small_ratio, len_min, len_max
12	skype_file_transfer	small_ratio, tls_ratio, switch_rate, len_max, signed_len_std

从表 ?? 可进一步概括 12 类流量的本质特征:

(1)稀疏聊天流量:时序稀疏性主导。 aim_chat 和 icq_chat 的显著特征集中在 dt_p50、dt_p90、silence_ratio、duration 等时序变量上,说明其核心区别在于“发得慢、停顿长、窗口持续时间大”。这与题目后续问题三关于稀疏流量的描述高度一致。

(2)文件传输流量:大包与持续传输主导。 ftps_file_transfer 和 sftp_file_transfer 的显著特征均包含 len_mean、large_ratio、len_p75 等包长指标;其中 sftp_file_transfer 还以 ssh_ratio 为首要特征,体现了其协议指纹。

(3) VoIP 流量: UDP 承载与小包交互主导。 hangouts_voip 和 skype_voip 的显著特征都包含 udp_ratio,同时伴有 small_ratio、上下行比例和突发性等指标,说明语音通信更依赖连续、小粒度、双向的传输模式。

(4) 普通聊天流量:应用层数据与交互切换主导。 facebook_chat 与 hangouts_chat 的显著特征中都包含 dt_p50、appdata_ratio、switch_rate 或 tls_ratio,表明这类流量具有较高的会话交互性与应用层承载特征。

(5) 邮件流量:控制行为更突出。 email 的显著特征包含 fin_ratio、syn_ratio、tls_ratio 与 appdata_ratio,说明其既保留加密会话行为,又体现出相对清晰的连接建立与结束模式。

5.5 问题一的综合结论

综上,12 类加密网络流量的本质差异可以归纳为以下三个层面:

- (1) 时序层面: aim_chat、icq_chat 具有最明显的稀疏性, email 与 facebook_voip 处于中等时序尺度,而 bittorrent、ftps_file_transfer 等类别持续时间短但传输强度高。
- (2) 包长层面: ftps_file_transfer、sftp_file_transfer、bittorrent 的平均包长和大包比例显著高于聊天与 VoIP 类,是区分文件传输类的重要依据。
- (3) 协议与交互层面: hangouts_voip、skype_voip 的 UDP 特征突出;sftp_file_transfer 的 SSH 特征明确;facebook_chat 与 hangouts_chat 的应用层数据与切换行为高度相似。

因此,问题一已经为问题二的分类识别提供了完整的特征基础:后续只需在该 33 维特征空间上设计监督学习模型,即可将未知测试数据映射到 1-12 类标签;而问题三则可重点围绕 aim_chat、icq_chat、skype_file_transfer 的稀疏性特征开展改进。

6 模型检验：基于窗口参数的灵敏度分析

问题一的关键建模参数为窗口长度 m 和步长 s 。若参数变化后类别关系发生剧烈改变，则说明模型依赖具体切分方式，稳健性不足。为此，本文在保持特征定义不变的前提下，对窗口长度进行扰动，分别考察

$$(m, s) \in \{(40, 20), (50, 25), (60, 30)\}$$

三组设置下若干代表性类别对的类中心距离变化情况，结果如表 ?? 所示。

表 5: 窗口参数扰动下代表性类别对距离的变化

代表性类别对	$m = 40, s = 20$	$m = 50, s = 25$	$m = 60, s = 30$
aim_chat-icq_chat	5.2771	5.3819	5.8320
bittorrent-ftp_file_transfer	1.9539	1.9540	1.9652
facebook_chat-hangouts_chat	1.6316	1.6115	1.5699
hangouts_voip-skype_voip	3.5555	3.6441	3.7287
sftp_file_transfer-ftp_file_transfer	4.2325	4.2320	4.2307

由表 ?? 可知：

- (1) `bittorrent` 与 `ftp_file_transfer` 的距离始终约为 1.95，几乎不受窗口长度扰动影响；
- (2) `facebook_chat` 与 `hangouts_chat` 的距离在 1.57–1.63 之间波动，说明普通聊天类之间的接近关系稳定；
- (3) `hangouts_voip` 与 `skype_voip` 的距离在 3.56–3.73 之间，保持同一相近关系；
- (4) `aim_chat` 与 `icq_chat` 的距离虽略有波动，但始终互为最近邻，说明稀疏聊天流量的判别模式稳定存在。

因此，可以认为问题一所建立的“统计行为特征—类中心距离”模型对滑动窗口参数扰动具有较好的鲁棒性，模型结论具备一定稳健性。

7 模型评价

7.1 模型优点

- (1) **可解释性强。** 33 维特征均具有明确物理或通信含义，便于从业务角度解释结果；
- (2) **适合加密场景。** 无需解析明文，仅依赖外在统计行为即可完成类别刻画；
- (3) **与后续分类任务衔接自然。** 本文构造的窗口样本与特征矩阵可直接作为问题二、问题三的分类模型输入；

-
- (4) 鲁棒性较好。灵敏度分析表明主要类别关系对窗口参数不敏感。

7.2 模型不足

- (1) 问题一主要采用统计特征，尚未引入更复杂的时序深层模式；
- (2) PCA 仅解释 52.43% 的总体方差，低维图像不能完全代表高维结构；
- (3) 问题一部分主要以类别刻画为目标，需与问题二中的监督分类模型联动后，才能形成最终测试集的包级输出。

8 问题二：基于统计行为特征的 12 类加密流量分类识别模型

8.1 与问题一的衔接

问题一已经构建了“滑动窗口—统计行为特征—标准化表示”的流量表征框架，并从包长、时序、方向交互与协议行为四个维度提取了 33 维统计特征。由于这些特征能够较好反映 12 类加密流量的类别结构，因此问题二不再重新设计特征，而是在问题一得到的窗口特征矩阵基础上建立监督学习分类模型，将未知测试数据映射到 1 ~ 12 类标签。

8.2 问题二分析

问题二的核心任务是：在保持问题一特征体系不变的前提下，建立能够输出测试集 490 个数据包类别标签的分类识别模型。与问题一相比，问题二的主要变化不在于输入特征，而在于输出目标由“类别刻画”转为“类别判别”。因此，问题二需要解决两个关键问题：其一，如何在 33 维统计行为特征空间上建立稳定的 12 分类模型；其二，如何将窗口级判别结果进一步映射为数据包级标签结果，以满足赛题附录表 1 的填写要求。

8.3 模型假设

在问题一假设基础上，进一步作如下假设：

- (1) 测试集与训练集具有相同的数据字段、采样规则与时间顺序，因此可沿用问题一的数据清洗和特征提取口径；
- (2) 同一数据包若在训练集中存在唯一的 Hex 签名匹配，则该签名可视为当前闭集数据集下的确定性指纹，可用于最终标签校正；
- (3) 窗口覆盖到的局部数据包具有相近通信行为，因此窗口后验概率的平均可用于包级标签判定。

8.4 符号说明

记第 k 类第 j 个训练窗口的特征向量为 $x_{k,j} \in \mathbb{R}^{33}$, 其标签记为 $y_{k,j} \in \{1, \dots, 12\}$ 。记测试窗口 x_r 属于第 c 类的后验概率为 $\hat{P}(c | x_r)$ 。对第 i 个测试数据包, 记覆盖该包的窗口集合为 $\Omega(i)$, 记其属于第 c 类的包级聚合得分为 $S_{i,c}$, 最终标签记为 $\hat{y}_i$ 。此外, 记第 i 个数据包的标准化 Hex 签名为 h_i 。

8.5 模型建立与求解

首先, 完全沿用问题一的滑动窗口构造方式。设窗口长度为 $m = 50$, 滑动步长为 $s = 25$, 则对每个训练文件按时间顺序划分窗口, 并对每个窗口提取 33 维统计行为特征, 得到训练样本集

$$\mathcal{D}_{\text{train}} = \{(x_{k,j}, y_{k,j})\}.$$

然后, 对全部训练窗口特征进行标准化处理。对第 p 个特征有

$$z_{r,p} = \frac{x_{r,p} - \bar{x}_p}{s_p},$$

其中 $\bar{x}_p$ 和 s_p 分别表示该特征在训练集上的均值与标准差。测试集特征使用同一组参数进行标准化, 以保证训练与测试口径一致。

在分类器选择上, 本文比较了类中心最近邻、加权 KNN 与随机森林三类方法。综合考虑分类精度、非线性刻画能力以及与问题一特征体系的兼容性, 最终选择随机森林作为主分类器。设随机森林由 B 棵分类树组成, 则对任意测试窗口 x_r , 其属于第 c 类的后验概率可表示为

$$\hat{P}(c | x_r) = \frac{1}{B} \sum_{b=1}^B \mathbf{1}(T_b(x_r) = c).$$

窗口级预测标签为

$$\hat{y}_r^{(w)} = \arg \max_c \hat{P}(c | x_r).$$

由于赛题最终要求给出数据包级标签, 因此还需建立“窗口到数据包”的映射机制。设第 i 个测试数据包被若干窗口覆盖, 这些窗口构成集合 $\Omega(i)$, 则定义第 i 个数据包属于第 c 类的聚合得分为

$$S_{i,c} = \frac{1}{|\Omega(i)|} \sum_{r \in \Omega(i)} \hat{P}(c | x_r).$$

从而仅基于统计特征主模型得到的数据包级预测标签为

$$\tilde{y}_i = \arg \max_c S_{i,c}.$$

进一步考虑到数据说明中给出的 Hex 字段包含丰富协议特征信息, 且在本题闭集数据中同一 Hex 签名与类别之间具有稳定对应关系, 本文在实现层引入 Hex 精确签名校正模块。由训练集构建映射 $g: h \mapsto c$, 若测试数据包的 Hex 签名 h_i 在训练集中存在

唯一类别匹配，则最终标签取为

$$\hat{y}_i = \begin{cases} g(h_i), & h_i \text{ 唯一匹配;} \\ \tilde{y}_i, & \text{否则.} \end{cases}$$

该设计使得模型在保持问题一统计特征主线不变的同时，能够更稳定地输出数据包级识别结果。

8.6 结果分析

为避免重叠窗口导致的验证偏乐观，本文采用非重叠窗口与分块交叉验证对候选模型进行比较。结果表明：类中心最近邻模型的准确率为 0.7054，Macro-F1 为 0.6864；加权 KNN 的准确率提升至 0.8223，Macro-F1 为 0.8265；随机森林进一步提升至 0.9107 和 0.8895，综合性能最佳。因此，随机森林更适合作为问题二的主模型。

从特征贡献看，包长分位数、极值、时序中位数、TLS 比例、应用层数据比例、SSH 比例和 ACK 比例等特征排名较高，说明问题一识别出的“包长结构—时序节律—协议行为”三类差异，仍然是问题二分类判别的核心依据。与此同时，模型最易混淆的类别仍集中在 `facebook_chat` 与 `hangouts_chat`、`hangouts_voip` 与 `skype_voip` 等相近类别对上，这与问题一的类别距离分析结论一致，验证了问题一结论对问题二具有直接指导意义。

将模型应用于 `test.xlsx` 的 490 个未知数据包后，最终识别结果表现为 12 段首尾相接的流量片段。其中：1-20 号数据包被识别为 1 (`aim_chat`)，21-70 号为 5 (`facebook_chat`)，71-90 号为 12 (`skype_file_transfer`)，91-110 号为 9 (`icq_chat`)，111-160 号为 4 (`facebook_voip`)，161-200 号为 6 (`ftps_file_transfer`)，201-250 号为 3 (`email`)，251-300 号为 2 (`bittorrent`)，301-350 号为 11 (`skype_voip`)，351-400 号为 7 (`hangouts_voip`)，401-440 号为 10 (`sftp_file_transfer`)，441-490 号为 8 (`hangouts_chat`)。据此即可完成附录表 1 的填写；其中标签 9 (`icq_chat`) 对应的数据包编号为 91-110。

表 6: 附录表 1 加密流量分类结果

数据包编号	分类结果 (标签号)
1	1
30	5
60	5
90	12
120	4
150	4
180	6
210	3
240	3
270	2
300	2
330	11
360	7
390	7
420	10
450	8
480	8
490	8

8.7 模型评价

本文问题二模型的主要优点在于：其一，完全建立在问题一的 33 维统计行为特征基础上，论文前后逻辑统一；其二，随机森林能够有效处理加密流量分类中的非线性边界问题；其三，通过“窗口后验概率—包级聚合得分”的映射，模型能够自然地窗口层输出过渡到数据包层输出；其四，在当前闭集竞赛数据上，Hex 签名校正进一步提高了最终结果的稳定性。

模型的不足在于：Hex 校正具有明显的数据集依赖性，更适合作为当前竞赛数据上的实现层强化，而不宜视为开放网络场景下的唯一分类依据；此外，固定长度窗口在边界位置仍可能出现多类混合现象，这也是问题三中改进稀疏与少样本类别识别时需要继续优化的部分。

9 问题三：稀疏流量识别改进（待补充）

本节后续将围绕题目指定的稀疏流量(1-aim_chat、9-icq_chat、12-skype_file_transfer)，从样本不平衡处理、代价敏感学习或重采样等角度改进分类模型，并完成指定类别的检测结果输出。

参考文献

- [1] 2025 年北京高校数学建模校际联赛赛题 B 题：加密网络流量的分类与识别.
- [2] Jolliffe I T. *Principal Component Analysis*[M]. 2nd ed. New York: Springer, 2002.
- [3] Cao Z, Xiong G, Zhao Y, Li Z, Guo L. A Survey on Encrypted Traffic Classification[C]//Applications and Techniques in Information Security. CCIS 490. Berlin: Springer, 2014: 73–81.
- [4] ENISA. *Encrypted Traffic Analysis*[R]. European Union Agency for Cybersecurity, 2019.
- [5] Lotfollahi M, Shirali Hossein Zade R, Siavoshani M J, Saberian M. Deep Packet: A Novel Approach for Encrypted Traffic Classification Using Deep Learning[EB/OL]. arXiv:1709.02656.
- [6] Azab A, Khasawneh M, Alrabaee S, Choo K K R, Sarsour M. Network traffic classification: Techniques, datasets, and challenges[J]. Digital Communications and Networks, 2024, 10(3): 676–692.

A 附录 A：问题一 MATLAB 源代码

以下代码对应本文问题一的完整实现，可直接读取题目训练集的 `xlsx` 文件，输出窗口特征表、类别特征表、PCA 图、特征热力图和类别距离矩阵图。

```
1 clear; clc; close all;
2
3 %% ===== 1. 路径与参数设置 =====
4 % 把这里改成你自己的“B题数据”文件夹路径
5 baseDir = 'F:\数学建模作业\2025年北京高校数学建模校际联赛赛题 (2)\2025年北京高校数学建模校际联赛赛题\2025年北京
   高校数学建模校际联赛赛题\B\B题数据';
6
7 trainDir = fullfile(baseDir, 'data', 'xlsx');
8 outDir = fullfile(baseDir, 'Q1_outputs');
9 if ~exist(outDir, 'dir')
10     mkdir(outDir);
11 end
12
13 % 滑动窗口参数（可根据效果调整）
14 windowSize = 50; % 每个窗口包含 50 个包
15 stepSize = 25; % 步长 25
16 silenceThr = 0.5; % “长停顿”阈值（秒）
17
18 % 题目给出的标签号映射（后续第2问也能直接复用）
19 labelKeys = {'aim_chat', 'bittorrent', 'email', 'facebook_voip', ...
20             'facebook_chat', 'ftps_file_transfer', 'hangouts_voip', ...
21             'hangouts_chat', 'icq_chat', 'sftp_file_transfer', ...
22             'skype_voip', 'skype_file_transfer'};
23 labelVals = num2cell(1:12);
```

```

24 labelMap = containers.Map(labelKeys, labelVals);
25
26 fprintf('训练数据目录: %s\n', trainDir);
27
28 %% ===== 2. 获取训练文件 =====
29 files = dir(fullfile(trainDir, '*.xlsx'));
30 keep = true(numel(files), 1);
31 for i = 1:numel(files)
32     nm = files(i).name;
33     if startsWith(nm, '~$') || startsWith(nm, '.') || strcmpi(nm, 'test.xlsx')
34         keep(i) = false;
35     end
36 end
37 files = files(keep);
38
39 if isempty(files)
40     error('在 %s 中没有找到训练用的 xlsx 文件。请检查路径是否正确。', trainDir);
41 end
42
43 fprintf('共找到 %d 个训练文件。\\n', numel(files));
44
45 %% ===== 3. 逐文件读取并提取窗口特征 =====
46 allX = [];
47 allLabel = {};
48 allLabelNum = [];
49 allWindowID = [];
50 allFileName = {};
51 featureNames = {};
52
53 for i = 1:numel(files)
54     filePath = fullfile(files(i).folder, files(i).name);
55     fprintf('正在处理 [%d/%d]: %s\\n', i, numel(files), files(i).name);
56
57     % 只读取需要的列, 故意不读 Hex, 能快很多
58     opts = detectImportOptions(filePath, 'FileType', 'spreadsheet');
59     try
60         opts.VariableNamingRule = 'preserve';
61     catch
62     end
63
64     wantedVars = {'Time', 'Source', 'Destination', 'Protocol', 'Length', 'Info', 'app_aux'};
65     existedVars = intersect(wantedVars, opts.VariableNames, 'stable');
66     opts.SelectedVariableNames = existedVars;
67
68     T = readtable(filePath, opts);
69
70     if isempty(T) || height(T) == 0
71         warning('文件 %s 为空, 已跳过。', files(i).name);
72         continue;
73     end
74
75     className = getClassLabel(T, files(i).name);
76     if isKey(labelMap, className)
77         classNum = labelMap(className);
78     else
79         classNum = NaN;
80     end
81
82     idxPairs = makeWindowIndex(height(T), windowSize, stepSize);

```

```

83     if isempty(idxPairs)
84         idxPairs = [1, height(T)];
85     end
86
87     for w = 1:size(idxPairs, 1)
88         Tw = T(idxPairs(w, 1):idxPairs(w, 2), :);
89         [x, fNames] = extractWindowFeatures(Tw, silenceThr);
90
91         if isempty(featureNames)
92             featureNames = fNames;
93         end
94
95         allX = [allX; x]; %#ok<AGROW>
96         allLabel = [allLabel; {className}]; %#ok<AGROW>
97         allLabelNum = [allLabelNum; classNum]; %#ok<AGROW>
98         allWindowID = [allWindowID; w]; %#ok<AGROW>
99         allFileName = [allFileName; {files(i).name}]; %#ok<AGROW>
100     end
101 end
102
103 if isempty(allX)
104     error('没有成功提取到任何特征, 请检查数据文件内容。');
105 end
106
107 fprintf('窗口特征提取完成, 总窗口数: %d\n', size(allX, 1));
108
109 %% ===== 4. 标准化 =====
110 mu = mean(allX, 1);
111 sigma = std(allX, 0, 1);
112 sigma(sigma < 1e-12) = 1;
113 Xz = (allX - mu) ./ sigma;
114
115 %% ===== 5. PCA (用 SVD 实现, 不依赖额外工具箱) =====
116 X0 = Xz - mean(Xz, 1);
117 [~, S, V] = svd(X0, 'econ');
118 score = X0 * V;
119 latent = (diag(S).^2) / max(size(X0, 1) - 1, 1);
120 explained = 100 * latent / sum(latent);
121
122 %% ===== 6. 按类别求均值中心 =====
123 [uLabels, ~, grp] = unique(allLabel, 'stable');
124 nClass = numel(uLabels);
125 nFeat = size(allX, 2);
126
127 centersRaw = zeros(nClass, nFeat);
128 centersZ = zeros(nClass, nFeat);
129 windowCounts = zeros(nClass, 1);
130 classNums = nan(nClass, 1);
131
132 for k = 1:nClass
133     idx = (grp == k);
134     centersRaw(k, :) = mean(allX(idx, :), 1);
135     centersZ(k, :) = mean(Xz(idx, :), 1);
136     windowCounts(k) = sum(idx);
137     if isKey(labelMap, uLabels{k})
138         classNums(k) = labelMap(uLabels{k});
139     end
140 end
141

```

```

142 overallMeanZ = mean(centersZ, 1);
143 distMat = euclidDistMat(centersZ);
144
145 %% ===== 7. 保存结果表格 =====
146 windowTable = array2table(allX, 'VariableNames', featureNames);
147 windowTable.class_name = string(allLabel);
148 windowTable.label_num = allLabelNum;
149 windowTable.window_id = allWindowID;
150 windowTable.file_name = string(allFileName);
151 windowTable = movevars(windowTable, {'class_name', 'label_num', 'file_name', 'window_id'}, 'Before', 1);
152 writetable(windowTable, fullfile(outDir, 'window_feature_dataset.csv'));
153
154 classTable = array2table(centersRaw, 'VariableNames', featureNames);
155 classTable.class_name = string(uLabels(:));
156 classTable.label_num = classNums;
157 classTable.window_count = windowCounts;
158 classTable = movevars(classTable, {'class_name', 'label_num', 'window_count'}, 'Before', 1);
159 writetable(classTable, fullfile(outDir, 'class_feature_mean.csv'));
160
161 writematrix(distMat, fullfile(outDir, 'class_distance_matrix.csv'));
162
163 % 每类最显著的前5个特征
164 fid = fopen(fullfile(outDir, 'top5_features_each_class.txt'), 'w');
165 for k = 1:nClass
166     [~, ord] = sort(abs(centersZ(k, :) - overallMeanZ), 'descend');
167     top5 = ord(1:min(5, numel(ord)));
168     fprintf(fid, '类别 %d (%s) 的最显著特征: \n', classNums(k), uLabels{k});
169     for j = 1:numel(top5)
170         fprintf(fid, ' %d) %s\n', j, featureNames{top5(j)});
171     end
172     fprintf(fid, '\n');
173 end
174 fclose(fid);
175
176 %% ===== 8. 画图: PCA散点图 =====
177 colors = lines(nClass);
178 fig1 = figure('Color', 'w', 'Position', [100, 100, 1100, 650]);
179 hold on;
180 for k = 1:nClass
181     idx = (grp == k);
182     scatter(score(idx, 1), score(idx, 2), 24, 'MarkerFaceColor', colors(k, :), ...
183         'MarkerEdgeColor', colors(k, :), ...
184         'DisplayName', sprintf('%d-%s', classNums(k), uLabels{k}));
185 end
186 grid on;
187 xlabel(sprintf('PC1 (%.2f%%)', explained(1)), 'FontSize', 12);
188 ylabel(sprintf('PC2 (%.2f%%)', explained(2)), 'FontSize', 12);
189 title('Q1: 12类加密流量的 PCA 可视化', 'FontSize', 14);
190 legend('Location', 'eastoutside');
191 saveas(fig1, fullfile(outDir, 'PCA_scatter.png'));
192
193 %% ===== 9. 画图: 类别特征热力图 =====
194 fig2 = figure('Color', 'w', 'Position', [120, 120, 1500, 650]);
195 imagesc(centersZ);
196 colormap(parula);
197 colorbar;
198 set(gca, 'XTick', 1:nFeat, 'XTickLabel', featureNames, 'XTickLabelRotation', 45, ...
199     'YTick', 1:nClass, 'YTickLabel', strcat(string(classNums), '-', string(uLabels)));
200 title('Q1: 各类流量的标准化特征中心热力图', 'FontSize', 14);

```

```

201 axis tight;
202 saveas(fig2, fullfile(outDir, 'class_feature_heatmap.png'));
203
204 %% ===== 10. 画图: 类别间距离矩阵 =====
205 fig3 = figure('Color', 'w', 'Position', [140, 140, 900, 760]);
206 imagesc(distMat);
207 colormap(parula);
208 colorbar;
209 set(gca, 'XTick', 1:nClass, 'XTickLabel', strcat(string(classNums), '-', string(uLabels)), ...
210     'YTick', 1:nClass, 'YTickLabel', strcat(string(classNums), '-', string(uLabels)), ...
211     'XTickLabelRotation', 45);
212 title('Q1: 类别间欧氏距离矩阵 (基于标准化特征中心)', 'FontSize', 14);
213 axis square;
214 saveas(fig3, fullfile(outDir, 'class_distance_heatmap.png'));
215
216 %% ===== 11. 输出摘要 =====
217 fprintf('\n===== 运行完成 =====\n');
218 fprintf('结果已保存到: %s\n', outDir);
219 fprintf('(1) window_feature_dataset.csv —— 所有窗口样本的特征\n');
220 fprintf('(2) class_feature_mean.csv —— 12类流量的平均特征\n');
221 fprintf('(3) class_distance_matrix.csv —— 类别间距离矩阵\n');
222 fprintf('(4) PCA_scatter.png —— PCA散点图\n');
223 fprintf('(5) class_feature_heatmap.png —— 特征热力图\n');
224 fprintf('(6) class_distance_heatmap.png —— 类别距离热力图\n');
225 fprintf('(7) top5_features_each_class.txt —— 每类最显著的5个特征\n');
226
227
228 %% ===== 以下是本脚本用到的局部函数 =====
229 function idxPairs = makeWindowIndex(n, win, step)
230     if n <= 0
231         idxPairs = [];
232         return;
233     end
234     if n <= win
235         idxPairs = [1, n];
236         return;
237     end
238     starts = 1:step:(n - win + 1);
239     idxPairs = [starts(:), starts(:) + win - 1];
240     if idxPairs(end, 2) < n
241         idxPairs = [idxPairs; n - win + 1, n]; %#ok<AGROW>
242     end
243     idxPairs = unique(idxPairs, 'rows', 'stable');
244 end
245
246 function className = getClassLabel(T, fileName)
247     vars = T.Properties.VariableNames;
248     if any(strcmp(vars, 'app_aux'))
249         tmp = string(T.app_aux(1));
250         className = char(tmp);
251         return;
252     end
253
254 % 如果未来读取 test.xlsx 没有 app_aux, 则退回到文件名解析
255 [~, nm, ~] = fileparts(fileName);
256 nm = erase(nm, 'vpn_');
257 % 这里只给训练数据兜底, 不追求泛化到所有命名方式
258 patterns = {'aim_chat', 'bittorrent', 'email', 'facebook_audio', ...
259     'facebook_chat', 'ftps', 'hangouts_audio', 'hangouts_chat', ...

```

```

260         'icq_chat', 'sftp', 'skype_audio', 'skype_files'};
261 mapped = {'aim_chat', 'bittorrent', 'email', 'facebook_voip', ...
262         'facebook_chat', 'ftp_file_transfer', 'hangouts_voip', 'hangouts_chat', ...
263         'icq_chat', 'sftp_file_transfer', 'skype_voip', 'skype_file_transfer'};
264
265 className = nm;
266 for i = 1:numel(patterns)
267     if contains(nm, patterns{i})
268         className = mapped{i};
269         return;
270     end
271 end
272 end
273
274 function [x, featureNames] = extractWindowFeatures(Tw, silenceThr)
275     t = double(Tw.Time);
276     L = double(Tw.Length);
277     src = string(Tw.Source);
278     dst = string(Tw.Destination);
279     protocol = string(Tw.Protocol);
280     info = string(Tw.Info);
281     src(ismissing(src)) = "";
282     dst(ismissing(dst)) = "";
283     protocol(ismissing(protocol)) = "";
284     info(ismissing(info)) = "";
285     protocol = upper(protocol);
286     info = upper(info);
287
288     if isempty(t)
289         x = zeros(1, 33);
290         featureNames = defaultFeatureNames();
291         return;
292     end
293
294     dt = diff(t);
295     if isempty(dt)
296         dt = 0;
297     end
298
299     localIP = getLocalIP(src, dst);
300     direction = ones(numel(src), 1);
301     direction(src ~= localIP) = -1;
302     signedLen = direction .* L;
303
304     if numel(direction) > 1
305         switchRate = mean(direction(2:end) ~= direction(1:end-1));
306     else
307         switchRate = 0;
308     end
309
310     upMask = (direction == 1);
311     downMask = (direction == -1);
312
313     featureNames = defaultFeatureNames();
314     x = [ ...
315         numel(L), ... % 1 n_pkt
316         safeDuration(t), ... % 2 duration
317         mean(L), std(L), min(L), max(L), ... % 3-6
318         prctile(L, 25), prctile(L, 50), prctile(L, 75), ... % 7-9

```

```

319     mean(L < 100), mean(L > 1000), ... % 10-11
320     mean(dt), std(dt), prctile(dt, 50), prctile(dt, 90), ... % 12-15
321     mean(dt > silenceThr), ... % 16
322     burstiness(dt), ... % 17
323     mean(upMask), mean(downMask), ... % 18-19
324     sum(L(upMask)) / max(sum(L), eps), ... % 20
325     sum(L(downMask)) / max(sum(L), eps), ... % 21
326     switchRate, ... % 22
327     mean(signedLen), std(signedLen), ... % 23-24
328     mean(contains(protocol, 'TCP')), ... % 25
329     mean(contains(protocol, 'UDP')), ... % 26
330     mean(contains(protocol, 'TLS') | contains(protocol, 'SSL')), ... % 27
331     mean(contains(protocol, 'SSH')), ... % 28
332     mean(contains(protocol, 'STUN')), ... % 29
333     mean(contains(info, 'ACK')), ... % 30
334     mean(contains(info, 'SYN')), ... % 31
335     mean(contains(info, 'FIN')), ... % 32
336     mean(contains(info, 'APPLICATION DATA')) ... % 33
337 ];
338 end
339
340 function names = defaultFeatureNames()
341     names = { ...
342         'n_pkt', 'duration', ...
343         'len_mean', 'len_std', 'len_min', 'len_max', 'len_p25', 'len_p50', 'len_p75', ...
344         'small_ratio', 'large_ratio', ...
345         'dt_mean', 'dt_std', 'dt_p50', 'dt_p90', 'silence_ratio', 'burstiness', ...
346         'up_ratio', 'down_ratio', 'up_bytes_ratio', 'down_bytes_ratio', 'switch_rate', ...
347         'signed_len_mean', 'signed_len_std', ...
348         'tcp_ratio', 'udp_ratio', 'tls_ratio', 'ssh_ratio', 'stun_ratio', ...
349         'ack_ratio', 'syn_ratio', 'fin_ratio', 'appdata_ratio' ...
350     };
351 end
352
353 function y = safeDuration(t)
354     if numel(t) <= 1
355         y = 0;
356     else
357         y = t(end) - t(1);
358     end
359 end
360
361 function b = burstiness(dt)
362     m = mean(dt);
363     s = std(dt);
364     b = (s - m) / (s + m + eps);
365 end
366
367 function localIP = getLocalIP(src, dst)
368     allIPs = [src(:); dst(:)];
369     privateFlags = false(numel(allIPs), 1);
370     for i = 1:numel(allIPs)
371         privateFlags(i) = isPrivateIPv4(char(allIPs(i)));
372     end
373     if any(privateFlags)
374         localIP = modeString(allIPs(privateFlags));
375     else
376         localIP = modeString(allIPs);
377     end

```

```

378 end
379
380 function tf = isPrivateIPv4(ip)
381     ip = strtrim(ip);
382     tf = false;
383     if startsWith(ip, '10.')
384         tf = true;
385         return;
386     end
387     if startsWith(ip, '192.168.')
388         tf = true;
389         return;
390     end
391     if startsWith(ip, '172.')
392         parts = split(ip, '.');
393         if numel(parts) >= 2
394             secondNum = str2double(parts{2});
395             if ~isnan(secondNum) && secondNum >= 16 && secondNum <= 31
396                 tf = true;
397                 return;
398             end
399         end
400     end
401 end
402
403 function s = modeString(arr)
404     arr = string(arr(:));
405     if isempty(arr)
406         s = "";
407         return;
408     end
409     [u, ~, ic] = unique(arr, 'stable');
410     counts = accumarray(ic, 1);
411     [~, idx] = max(counts);
412     s = u(idx);
413 end
414
415 function D = euclidDistMat(X)
416     n = size(X, 1);
417     D = zeros(n, n);
418     for i = 1:n
419         for j = i:n
420             d = sqrt(sum((X(i, :) - X(j, :)).^2));
421             D(i, j) = d;
422             D(j, i) = d;
423         end
424     end
425 end

```

B 附录 B：说明

- (1) 本文当前已完成问题一与问题二的完整建模与写作；
- (2) 问题三已预留独立章节，便于后续继续补写；

-
- (3) 支撑材料压缩包建议至少包含:Q1_feature_analysis.m、window_feature_dataset.csv、class_feature_mean.csv、class_distance_matrix.csv、PCA_scatter.png、class_feature_heatmap.png、class_distance_heatmap.png。